our case, the JWT is an Access Token, and we want to configure how to build `GrantedAuthority` objects from the values in the `roles` claim. In the Catalog Service project (catalog-service), open the `SecurityConfig` class and define a new `JwtAuthenticationConverter` bean.

Listing 12.9 Mapping the roles from the JWT to granted authorities

```
@EnableWebSecurity
public class SecurityConfig {
  ...

  @Bean
  public JwtAuthenticationConverter jwtAuthenticationConverter() {
    var jwtGrantedAuthoritiesConverter =
      new JwtGrantedAuthoritiesConverter();                    ❶
    jwtGrantedAuthoritiesConverter
      .setAuthorityPrefix("ROLE_");                            ❷
    jwtGrantedAuthoritiesConverter
      .setAuthoritiesClaimName("roles");                       ❸

    var jwtAuthenticationConverter =
      new JwtAuthenticationConverter();                        ❹
    jwtAuthenticationConverter
      .setJwtGrantedAuthoritiesConverter(jwtGrantedAuthoritiesConverter);
    return jwtAuthenticationConverter;
  }
}
```

❶ Defines a converter to map claims to GrantedAuthority objects

❷ Applies the “ROLE_” prefix to each user role

❸ Extracts the list of roles from the roles claim

❹ Defines a strategy to convert a JWT. We’ll only customize how to build granted authorities out of it.

With this bean in place, Spring Security will associate a list of `GrantedAuthority` objects with each authenticated user, and we can use them to define authorization policies.